

Objective

This lab has three parts. First, students write plain analytical SQL queries over a student CGPA table — no materialized views yet, just GROUP BY practice at increasing levels of granularity. Second, students rebuild a subset of those answers as **materialized views** with **derived attributes** — values computed from columns stored inside the view itself, including one derived by joining two materialized views together. Third, students gain practical experience in working with APIs and integrating external data into databases: fetching research article metadata from the Semantic Scholar API, storing it in SQL, extending that metadata with citation references, and performing analytical queries on it — developing hands-on expertise in combining programming with database operations, a key skill in Data Warehousing and Business Intelligence.

Case Study: FCIT Academic Performance

Background.

The Faculty of Computer Science & Information Technology (FCIT) wants a clearer picture of student performance across its departments, shifts, and batches. The accompanying schema-and-data-load script creates one table, **FCIT\_Students**, with about 200 students spread across four departments, two shifts, and four batches, each with a CGPA. The faculty office wants both quick ad-hoc queries and a small set of stored, derived summaries that answer “how does this department compare to FCIT overall” without recomputing everything from scratch each time.

FCIT\_STUDENTS

| Column      | Type          | Description                                                                     |
|-------------|---------------|---------------------------------------------------------------------------------|
| StudentID   | NUMBER (PK)   | Unique student identifier                                                       |
| StudentName | VARCHAR2(100) | Student name                                                                    |
| Department  | VARCHAR2(50)  | Computer Science, Data Science, Software Engineering, or Information Technology |
| Shift       | VARCHAR2(10)  | Morning or Evening                                                              |
| Batch       | VARCHAR2(10)  | F22, F23, F24, or F25                                                           |
| CGPA        | NUMBER(3,2)   | 0.00 to 4.00                                                                    |

Sample rows (illustrative — the load script generates the actual values)

| StudentID | StudentName | Department           | Shift   | Batch | CGPA |
|-----------|-------------|----------------------|---------|-------|------|
| 3001      | Student 001 | Data Science         | Morning | F23   | 3.45 |
| 3002      | Student 002 | Computer Science     | Evening | F24   | 2.80 |
| 3003      | Student 003 | Software Engineering | Morning | F22   | 3.90 |

*Note.* New rows inserted during the lab should use **StudentID** ≥ 4001 so they do not clash with the generated data.

Tasks

## Task 1 — Analytical Queries on FCIT\_Students

*Plain SQL, no materialized views — just GROUP BY at increasing levels of detail.*

- 1.1 Write a query to find the **overall average CGPA of FCIT** (a single value, across all students).
- 1.2 Write a query to find the **average CGPA for every department**.
- 1.3 Write a query to find the **average CGPA for every shift** (Morning vs. Evening).
- 1.4 Write a query to find the **average CGPA for every batch**.
- 1.5 Write a query to find the **average CGPA for every department, broken down by shift** (group by Department and Shift together).

## Task 2 — Materialized Views with Derived Attributes

*Each question asks you to store raw aggregates inside a materialized view, then derive a new value from those stored columns.*

- 2.1 **Simple.** Create a materialized view that stores, per department, the **total CGPA (SUM)** and the **student count**. Then write a query that derives the **average CGPA** from these two stored columns.
- 2.2 **Complex (single view).** Create a materialized view that stores, per department, the student count, the count of students with CGPA  $\geq 3.5$ , and the count of students with CGPA  $< 2.5$ . Then write a query that derives the **percentage of high performers** and the **percentage of at-risk students** per department from these stored counts.
- 2.3 **Join of two materialized views.** Create one materialized view that stores the **overall FCIT average CGPA**, and a second that stores the **average CGPA per department**. Then write a query that **joins the two** and derives each department's **deviation from the FCIT overall average**, showing which departments are above or below it.

## Task 3 — Connect to the Semantic Scholar API

*Fetch research-paper metadata with Python and store it in SQL.*

- 3.1 Request your own free API key at <https://www.semanticscholar.org/product/api> (click “Request an API key”) — it arrives by email. The API also works without a key at a lower rate limit, so you can start testing while the key is pending. Use Python's **requests** library to query the Semantic Scholar API for a topic (e.g., “Artificial Intelligence in Education”), sending your key in an **x-api-key** header once you have it. Example endpoint: <https://api.semanticscholar.org/graph/v1/paper/search?query=artificial+intelligence+in+education&limit=20&fields=title,authors,abstract,year,citationCount,referenceCount,references>. Parse and display the title, authors, abstract, year, citation count, and paper ID (DOI or Semantic Scholar ID) for each result.
- 3.2 Create a table **Articles\_API** with columns PaperID (primary key), Title, Authors, Abstract, Year, and CitationCount, and insert the papers fetched in Task 3.1 into it.

**3.3**

Add a column **Reference\_List** to `Articles_API` (to store DOIs or IDs of cited papers as JSON/text). For each paper, extract at least 2 references from the API response and store them.

***Submit:** your SQL script for Tasks 1 and 2 with query outputs, your Python script and a screenshot of the fetched data for Task 3, and the final contents of `Articles_API`, including the `Reference_List` column.*